

BIM INGENIEROS • BIMROCKET + IOT

Lección 1

Del sensor REST al objeto BIM conectado

Cómo llevar una medición IoT simulada hasta una sala BIM que muestra, colorea y valida el dato.

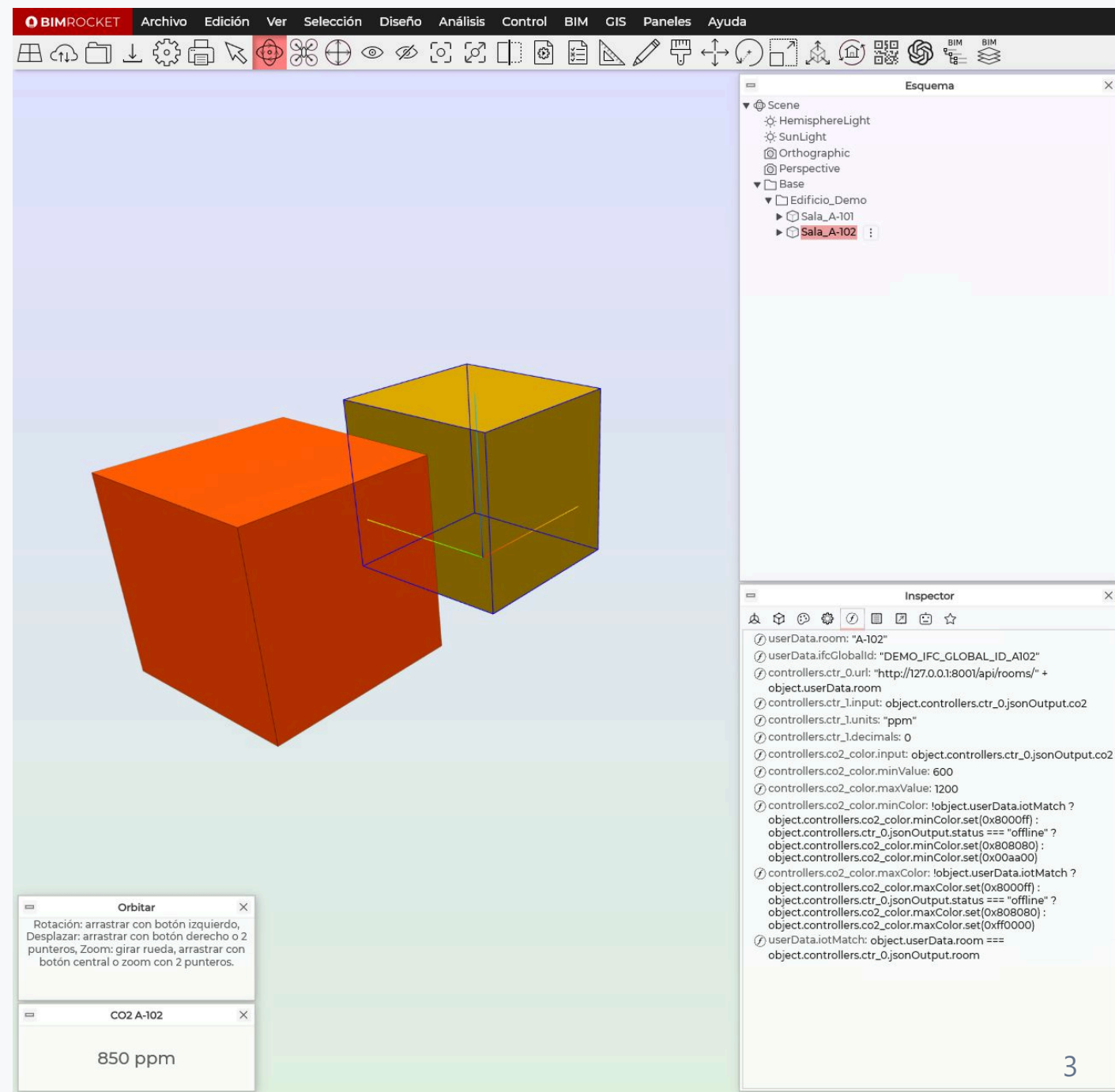
Objetivo de la lección

Entender el primer flujo completo:

```
sensor/API → dato JSON → objeto BIM → visualización → validación
```

Qué vas a construir

- API REST simulada con varias salas.
- Modelo BIMROCKET con `Sala_A-101` y `Sala_A-102`.
- Panel de CO₂ por sala.
- Color por CO₂, offline e identidad incorrecta.



El sensor devuelve JSON

Respuesta REST — Sala A-102

```
{  
  "room": "A-102",  
  "ifcGlobalId": "DEMO_IFC_GLOBAL_ID_A102",  
  "temperature": 22.5,  
  "co2": 815,  
  "status": "online"  
}
```

La API devuelve identidad, medición y estado.

Camino del dato

Camino del dato IoT en BIMROCKET



La geometría se convierte en un objeto con identidad, datos vivos y reglas de confianza.

Identidad BIM + IoT

La clave no es solo leer datos.

La clave es saber **a qué objeto BIM pertenecen.**

Cada sala sabe quién es

```
Sala_A-102
├─ userData.room = A-102
└─ userData.ifcGlobalId = DEMO_IFC_GLOBAL_ID_A102
```

`room` identifica la sala para la API.

`ifcGlobalId` representa el identificador BIM/IFC.

URL dinámica del sensor

URL dinámica del sensor

```
"http://127.0.0.1:8001/api/rooms/" + object.userData.room
```

Texto fijo + identidad de la sala actual

Sala_A-102 → /api/rooms/A-102

Sala_A-101 → /api/rooms/A-101

Fórmula para mostrar CO₂

Path:

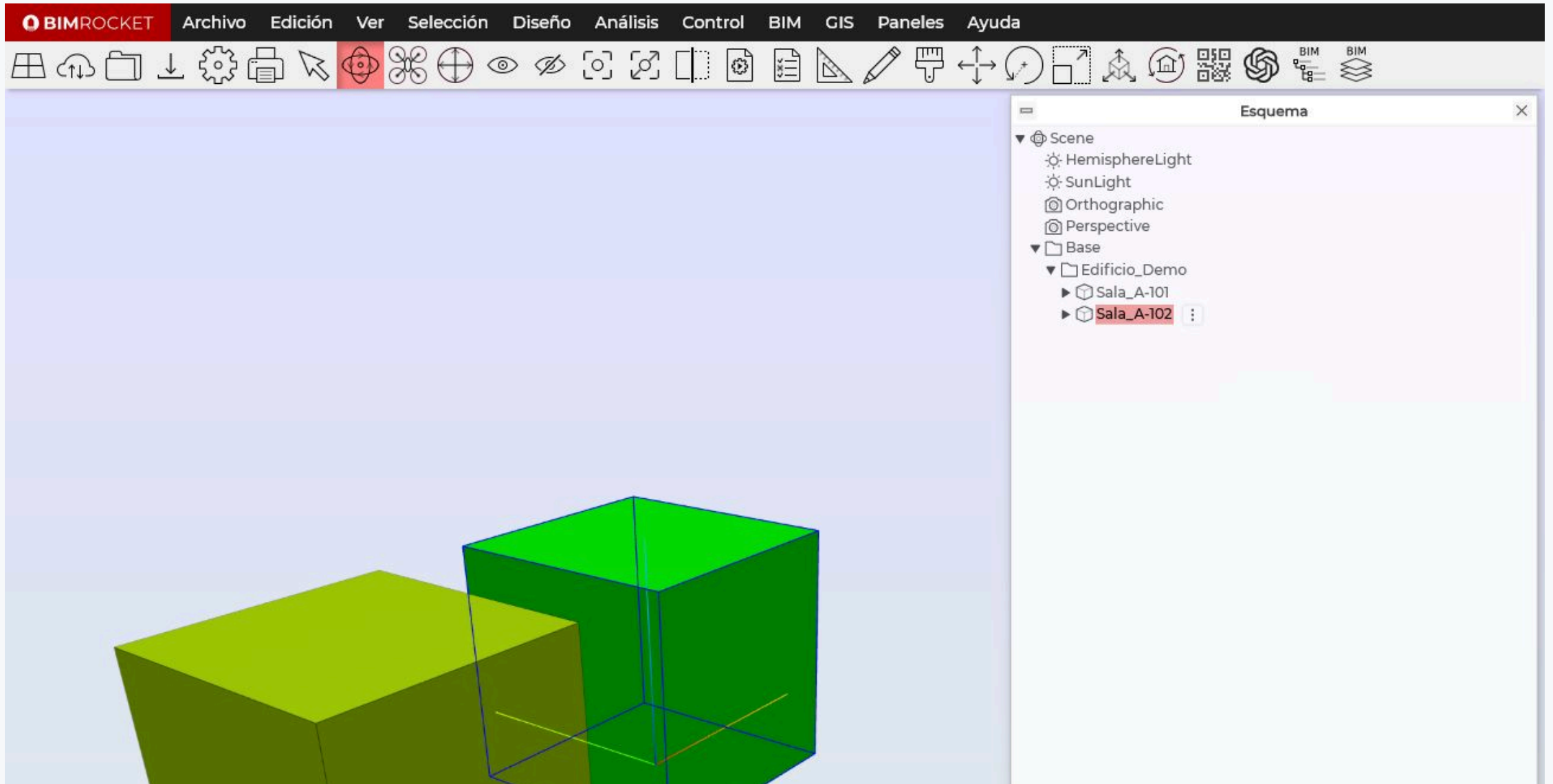
```
controllers.ctr_1.input
```

Expression:

```
object.controllers.ctr_0.jsonOutput.co2
```

El `DisplayController` muestra el CO₂ recibido por `RestPollController`.

En BIMROCKET: fórmulas



Validar confianza del dato

Un dato puede llegar, pero no pertenecer al objeto correcto.

iotMatch

Validación de identidad IoT

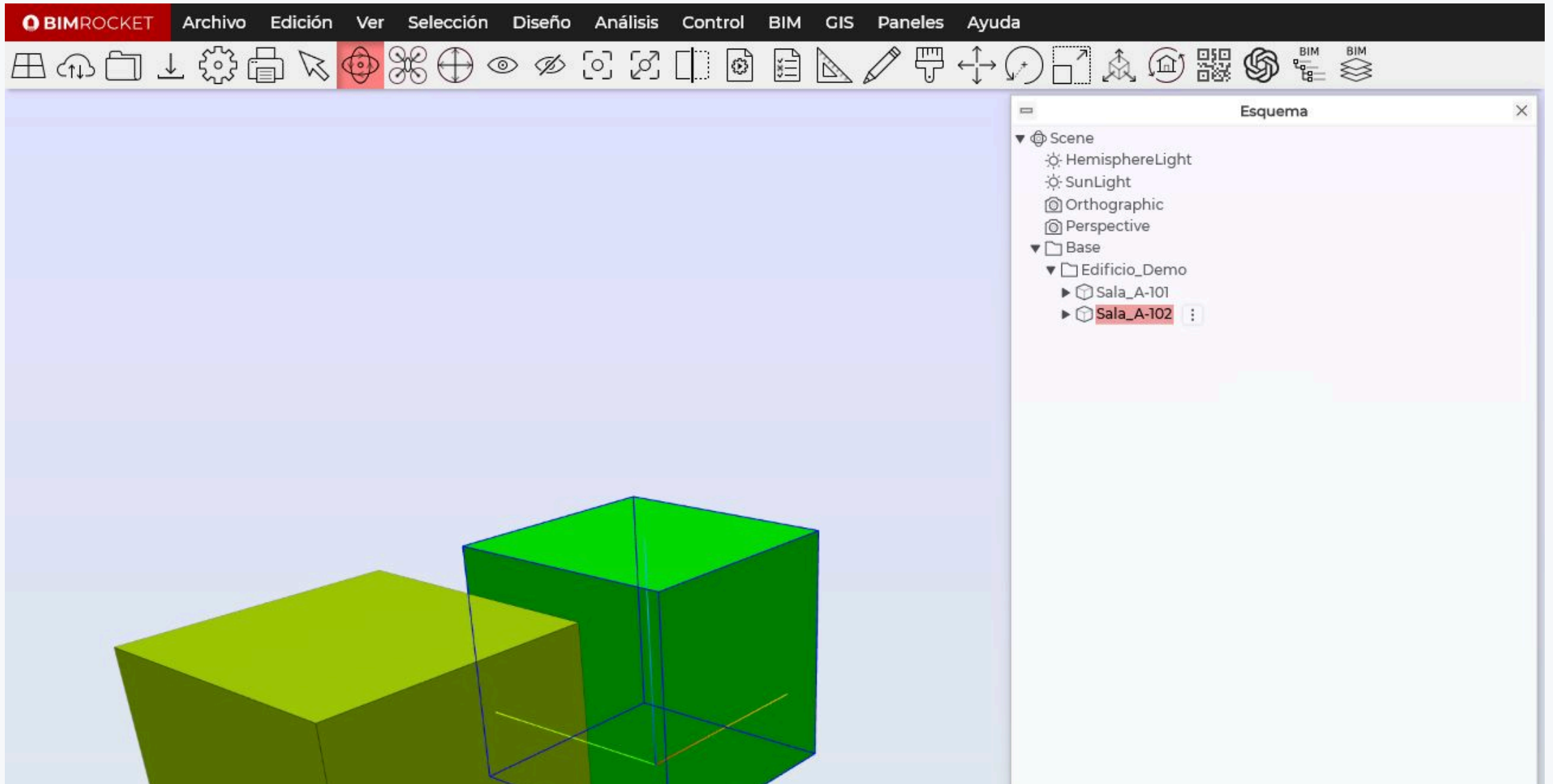
```
object.userData.room === object.controllers.ctr_0.jsonOutput.room
```

Pregunta:

¿La sala BIM coincide con la sala que dice la API?

true = dato confiable · false = alerta visual morada

En BIMROCKET: userData



Estados visuales

Estados visuales del dato

Todo correcto

`status = online`
`iotMatch = true`
color según CO₂

Sensor offline

`status = offline`
`iotMatch = true`
gris

Identidad incorrecta

`iotMatch = false`
morado
prioridad máxima

Prioridad: morado → gris → CO₂

Reglas finales de color

Para `minColor`:

```
!object.userData.iotMatch
  ? object.controllers.co2_color.minColor.set(0x8000ff)
  : object.controllers.ctr_0.jsonOutput.status === "offline"
    ? object.controllers.co2_color.minColor.set(0x808080)
    : object.controllers.co2_color.minColor.set(0x00aa00)
```

Prioridad de confianza

Prioridad	Condición	Color	Significado
1	<code>iotMatch = false</code>	Morado	Dato no confiable
2	<code>status = offline</code>	Gris	Sensor no disponible
3	Todo correcto	Verde-rojo	CO ₂ válido

Pruebas realizadas

No basta con configurar.
Hay que probar cada estado.

Prueba 1 — Caso normal

```
status = online  
iotMatch = true  
color = según CO2
```

Resultado: la sala usa la escala verde → rojo.

Prueba 2 — Sensor offline

URL temporal:

```
"http://127.0.0.1:8001/api/rooms/" + object.userData.room + "?offline=1"
```

Resultado esperado:

```
status = offline  
iotMatch = true  
color = gris
```

Prueba 3 — Identidad incorrecta

Fórmula temporal:

```
"A-999" === object.controllers.ctr_0.jsonOutput.room
```

Resultado esperado:

```
iotMatch = false  
color = morado
```

Modelo final

Archivo de referencia:

```
examples/bimrocket-models/lab-03-dos-salas-iot.brf
```

Contiene:

- dos salas conectadas;
- URL dinámica;
- paneles de CO₂;
- validación `iotMatch` ;
- estados visuales de confianza.

Cierre conceptual

Un gemelo digital no es solo un modelo 3D.
Es geometría + identidad + datos vivos + reglas de confianza.

BIM INGENIEROS

Siguiente paso

Del sensor simulado al sensor real con ESP32.

La lógica aprendida se mantiene. Solo cambia la fuente del dato.